

Run-Length Encoding

1. Problem Statement

Given a string `s`, implement a function `encode` that performs **Run-Length Encoding (RLE)**.

Run-Length Encoding compresses consecutive occurrences of the same character by replacing them with:

```
character + count
```

Example

Input:
aaaabbbccc

Output:
a4b3c3

Explanation:

```
aaaa → a4  
bbb  → b3  
ccc  → c3
```

2. Approach

We use a `StringBuilder` and a variable `count`.

Algorithm

1. Create an empty `StringBuilder`.
2. Initialize `count = 1`.
3. Start iterating from index `1`.
4. Compare the current character with the previous character.
5. If both are the same, increment `count`.
6. If they are different:
7. Add the previous character.

8. Add its count.
9. Reset `count` to `1`.
10. After the loop, add the last character and its count.
11. Return the encoded string.

Basic Idea

Same character

↓

`count++`

Different character

↓

`character + count`

↓

`count = 1`

3. Java Code

```
class Solution {
    public static String encode(String s) {

        StringBuilder sb = new StringBuilder();

        int count = 1;

        for (int i = 1; i < s.length(); i++) {

            if (s.charAt(i) == s.charAt(i - 1)) {
                count++;
            } else {
                sb.append(s.charAt(i - 1));
                sb.append(count);
                count = 1;
            }
        }

        // Add the last character and its count
        sb.append(s.charAt(s.length() - 1));
        sb.append(count);

        return sb.toString();
    }
}
```

```
}  
}
```

4. Code Explanation

StringBuilder

```
StringBuilder sb = new StringBuilder();
```

Used to build the final encoded string efficiently.

Count

```
int count = 1;
```

The first character is initially counted as one occurrence.

Loop

```
for (int i = 1; i < s.length(); i++)
```

The loop starts from index **1** because every character is compared with the character before it.

For:

```
aaaab
```

we compare:

```
s[1] with s[0]  
s[2] with s[1]  
s[3] with s[2]  
s[4] with s[3]
```

Same Characters

```
if (s.charAt(i) == s.charAt(i - 1)) {  
    count++;  
}
```

If the current and previous characters are the same, increase the count.

For:

aaaa

the count becomes:

1 → 2 → 3 → 4

Different Characters

```
else {  
    sb.append(s.charAt(i - 1));  
    sb.append(count);  
    count = 1;  
}
```

When a different character is found, the previous group is complete.

For:

aaaab

when **b** is found:

a != b

So we add:

a4

Then:

```
count = 1;
```

because `b` starts a new group.

5. Why Add the Last Character Separately?

These lines are important:

```
sb.append(s.charAt(s.length() - 1));  
sb.append(count);
```

The `else` block only executes when a **different character** is found.

The last group has no next character, so the `else` block will not process it.

Example

```
aaaabbbccc
```

The loop processes:

```
aaaa → a4  
bbb  → b3
```

But there is no character after `ccc`.

Therefore, we manually add:

```
c3
```

Final result:

```
a4b3c3
```

6. Dry Run

Consider:

```
s = "aaabbc"
```

Initial:

```
count = 1  
sb = ""
```

i	Current	Previous	Action	count	sb
1	a	a	Same → count++	2	""
2	a	a	Same → count++	3	""
3	b	a	Add a3	1	"a3"
4	b	b	Same → count++	2	"a3"
5	c	b	Add b2	1	"a3b2"

After the loop:

```
Add c1
```

Final output:

```
a3b2c1
```

7. Examples

Example 1

```
Input:  
aaaabbbccc
```

Output:
a4b3c3

Example 2

Input:
aabbbcccc

Output:
a2b3c4

Example 3

Input:
abcd

Output:
a1b1c1d1

Example 4

Input:
aaaaa

Output:
a5

8. Important Observation

Run-Length Encoding counts **consecutive** characters.

For:

ababab

the output is:

a1b1a1b1a1b1

It is **not**:

a3b3

because the `a` characters are not consecutive.

9. Complexity Analysis

Let `n` be the length of the string.

Time Complexity

The string is traversed once.

$O(n)$

Space Complexity

The `StringBuilder` stores the encoded result.

$O(n)$

Therefore:

Time Complexity : $O(n)$
Space Complexity : $O(n)$

10. Key Takeaway

The main logic is:

```
if (s.charAt(i) == s.charAt(i - 1)) {  
    count++;  
} else {  
    sb.append(s.charAt(i - 1));  
    sb.append(count);  
}
```



```
        count = 1;  
    }
```

The final group must be handled separately:

```
sb.append(s.charAt(s.length() - 1));  
sb.append(count);
```

Core DSA Pattern

```
Traverse String  
    ↓  
Compare Adjacent Characters  
    ↓  
Same → Increase Count  
    ↓  
Different → Store Character + Count  
    ↓  
Reset Count  
    ↓  
Process Last Group
```